

## test\_scenario\_kinds.vr

\$VCS\_HOME/doc/examples/nativetestbench/openvera/rvm/scenarios/test\_scenario\_kinds.vr

```
//
// Demonstrate how to use scenarios kinds
// i.e. each scenario defined as a different
// variations of a single descriptor
//
#include "scenarios.vrh"
//
// Telegraph good & bad cells
// good - BAD - good - BAD - good
// VPIa  VPIb  VPIa  VPIb  VPIa
//
class my_atm_cell_scenarios extends atm_cell_scenario {
    integer BAD_CELLS;
    integer PROCEDURAL;

    constraint bad_cells_scenario {
        if (kind == BAD_CELLS) {
            length == 5;
            repeated == 0;
            foreach (this.items, i) {
                if (i % 2) this.items[i].hec != 8'h00;
                else      this.items[i].hec == 8'h00;
                if (i > 1) this.items[i].vpi == this.items[i-2].vpi;
                if (i == 1) this.items[i].vpi != this.items[i-1].vpi;
            }
        }
    }

    constraint procedural_scenario {
        if (kind == PROCEDURAL) {
            length == 0;
            repeated == 0;
        }
    }

    constraint hec_valid {
        if (kind != this.BAD_CELLS) {
            foreach (this.items, i) {
```

```

        this.items[i].hec == 8'h00;
    }
}

task new() {
    super.new();
    this.BAD_CELLS = define_scenario("Telegraph bad cells", 5);
    this.PROCEDURAL = define_scenario("Procedural cells", 0);
}

task pre_randomize() {
    super.pre_randomize();
    foreach (this.items, i) {
        void = this.items[i].constraint_mode(OFF, "hec_valid");
    }
}

virtual function integer apply_t(atm_cell_channel channel) {
    if (kind == this.PROCEDURAL) {
        apply_t = this.procedural_scenario_t(channel);
        return;
    }
    apply_t = super.apply_t(channel);
}

function integer procedural_scenario_t(atm_cell_channel channel) {
    atm_cell cell = new;

    void = cell.randomize() with {
        gfc == 4'h0;
        vpi == 8'h00;
        vci == 16'h0000;
        pt == 3'h0;
        clp == 1'b0;
    };

    repeat (3) {
        atm_cell c;
        cast_assign(c, cell.copy());
        channel.put_t(c);
    }

    void = cell.randomize() with {
        gfc == 4'hF;

```

```

        vpi == 8'hFF;
        vci == 16'hFFFF;
        pt  == 3'h7;
        clp == 1'b1;
    };

    channel.put_t(cell);

    procedural_scenario_t = 4;
}
}

class my_atm_cell_scenario_gen_callbacks extends atm_cell_scenario_gen_callbacks{
    virtual task post_scenario_gen_t(atm_cell_scenario_gen gen,
                                     atm_cell_scenario    scenario,
                                     var bit               drop)

    {
        scenario.display();
        printf("-----\n");
    }
}

program test_gen {
    atm_cell_scenario_gen gen = new("Singleton");
    my_atm_cell_scenario_gen_callbacks cb = new;
    my_atm_cell_scenarios ss = new;
    gen.scenario_set[0] = ss;
    gen.register_callback(cb);
    // Drain the output channel
    fork
    {
        while (1) {
            atm_cell cell = gen.out_chan.get_t();
        }
    }
    join none
    rvm_note(gen.log, "Generating 25 cells or 10 scenarios...");
    gen.stop_after_n_insts = 25;
    gen.stop_after_n_scenarios = 10;
    gen.start_xactor();
    void = gen.notify.wait_for_t(gen.DONE);
    delay(100);
    gen.log.report();
}

```

